# UNLOX ACADEMY

VLSI Design Program, September 2026 Batch

# MINOR PROJECT REPORT

## PROJECT 03

## DESIGN AND SIMULATION OF A 4-BIT COUNTER

*Modelled in Verilog HDL as a Synchronous Up/Down Counter*



| | |
|---|---|
| **Student Name** | Thipperudra K T |
| **Roll No.** | 4SM23EE037 |
| **Date** | 24 October 2026 |
| **Instructor** | Kuldeep Sompura |

# Declaration

I, Thipperudra K T, a participant of the Unlox Academy VLSI Design program (September 2026 batch), hereby declare that this report titled "Design and Simulation of a 4-Bit Counter using Verilog HDL" has been prepared by me as part of the Minor Project (Project 03) of the program. The design, source code, testbench and simulation results described here have been studied, explained and documented in this report in my own words, as required by the project instructions.

The report has not been submitted to any other institution or program for any award or evaluation.

Date: 24 October 2026

Thipperudra K T

# Acknowledgement

I sincerely thank Unlox Academy for the VLSI Design program and for giving me the opportunity to work on a practical digital design project. The structured sessions on digital logic, Verilog HDL and simulation made it possible to understand how a basic building block such as a counter is described, verified and documented as hardware.

I am grateful to the instructor, Kuldeep Sompura, and the mentors and trainers of the program for their clear explanations of sequential circuits, counters and testbench writing, and for the assignment material that formed the basis of this project. I also thank my fellow participants for the discussions that helped me clarify several doubts, and my family for their constant support and encouragement.

Thipperudra K T

# Abstract

Counters are among the most fundamental building blocks of digital systems. They are used for counting events, measuring time, dividing frequencies, generating addresses and sequencing control logic. This project presents the design and simulation of a 4-bit counter in Verilog Hardware Description Language (HDL).

The design, named counter_4bit, is a synchronous counter built around a 4-bit register that is updated on the rising edge of a single clock. It has a synchronous active-high reset (rst), a count-enable input (en) and a direction input (up_down). When enabled, it counts up from 0 to 15 and rolls over to 0, or counts down from 15 to 0 and rolls over to 15. When disabled it holds its value, and reset forces the output to 0000 irrespective of the other inputs.

A testbench, tb_counter_4bit, generates a 10 ns clock and drives seven stimulus phases: reset, counting up with wrap-around, hold, counting down with wrap-around, direction change, reset in the middle of counting and resumption of counting. The report documents the theory, the function table, the count sequence, the source code, the testbench, the simulation waveforms, an edge-by-edge trace, a comparison of counter types and a verification summary showing that the outputs match the expected behaviour in every phase.

**Keywords:** Verilog HDL, 4-bit Counter, Synchronous Counter, Up/Down Counter, Modulo-16, Testbench, Simulation, Waveform.

# Table of Contents

**Declaration** . . . . . 2

**Acknowledgement** . . . . . 3

**Abstract** . . . . . 4

**Table of Contents** . . . . . 5

**1. Introduction** . . . . . 7

1.1 Background . . . . . 7

1.2 Motivation . . . . . 7

1.3 Scope of the Project . . . . . 7

1.4 Organisation of the Report . . . . . 7

**2. Problem Statement and Objectives** . . . . . 8

2.1 Problem Statement . . . . . 8

2.2 Objectives . . . . . 8

2.3 Design Requirements . . . . . 8

**3. Theoretical Background** . . . . . 9

3.1 Sequential Circuits and Counters . . . . . 9

3.2 Synchronous and Asynchronous Counters . . . . . 9

3.3 Modulo and Wrap-Around . . . . . 9

3.4 Hardware Description Language and Verilog . . . . . 9

3.5 Synchronous Reset . . . . . 10

3.6 Enable and Direction Control . . . . . 10

3.7 Bit-Level View of Binary Counting . . . . . 10

**4. Tools and Design Elements** . . . . . 11

4.1 Tools Used . . . . . 11

4.2 Signals of the Counter . . . . . 11

4.3 Function Table . . . . . 11

**5. Design of the 4-Bit Counter** . . . . . 12

5.1 Design Overview . . . . . 12

5.2 Block Diagram . . . . . 12

5.3 Count Sequence . . . . . 13

5.4 Source Code Explanation . . . . . 14

5.5 Timing Analysis . . . . . 14

5.6 Working Steps of the Counter . . . . . 14

5.7 Bit-Level Toggle Equations . . . . . 15

5.8 Corner Cases . . . . . 15

**6. Testbench and Simulation Results** . . . . . 17

6.1 Testbench . . . . . 17

6.2 Stimulus Timeline . . . . . 17

6.3 Simulation Waveform . . . 18
6.4 Console Output . . . 19
6.5 Edge-by-Edge Trace . . . 20
6.6 Verification Summary . . . 21
6.7 Summary of Results . . . 21
6.8 Bit-Level Waveform and Frequency Division . . . 22

**7. Comparison of Counter Types . . . 23**

**8. Advantages, Limitations, Applications and Future Scope . . . 24**

8.1 Advantages . . . 24
8.2 Limitations . . . 24
8.3 Applications . . . 24
8.4 Future Scope . . . 24

**9. Conclusion . . . 25**

**References . . . 25**

**Appendix A: Complete Source Code of the Counter . . . 26**

**Appendix B: Complete Testbench . . . 27**

**Appendix C: Glossary . . . 28**

**Appendix D: Complete Console Log . . . 29**

**Appendix E: Submission Checklist . . . 30**

**Appendix F: Simulation Procedure . . . 31**

F.1 Icarus Verilog with GTKWave . . . 31
F.2 Xilinx Vivado, ModelSim or Questa . . . 31
F.3 What to Check in the Waveform . . . 31

# 1. Introduction

## 1.1 Background

Digital systems keep track of time, events and position by counting. A clock edge arrives, a number is incremented, and that number then decides what the rest of the circuit does next. A **counter** is the sequential circuit that performs this operation. It stores its present value in flip-flops and moves to the next value on every active clock edge.

Modern digital chips such as FPGAs and ASICs are designed by describing circuits like this in a Hardware Description Language (HDL) such as Verilog or VHDL, simulating the description to check its behaviour, and finally synthesising it into gates and flip-flops. A 4-bit counter is the simplest complete example of that flow.

## 1.2 Motivation

The 4-bit counter is the classic first sequential design in VLSI. Although small, it brings together several important ideas: clocked registers, non-blocking assignments, synchronous reset, enable control, direction control, overflow and wrap-around behaviour, and testbench-based verification. The habits built while designing it carry over directly to larger designs such as timers, FIFOs, address generators and finite state machines.

## 1.3 Scope of the Project

This project covers the design and functional simulation of one 4-bit counter, written in Verilog HDL, which includes:

- A synchronous up/down counter with count enable and synchronous reset.
- A self-written testbench that generates the clock and applies reset, enable and direction stimulus.
- Waveform output demonstrating correct operation, together with an edge-by-edge trace.

Physical implementation on an FPGA board and logic synthesis reports are outside the scope of this project.

## 1.4 Organisation of the Report

Chapter 2 states the problem and objectives. Chapter 3 gives the theoretical background. Chapter 4 lists the tools, signals and the function table. Chapter 5 describes the counter design and explains the source code. Chapter 6 presents the testbench and simulation results. Chapter 7 compares counter types. Chapter 8 discusses advantages, limitations, applications and future scope, followed by the conclusion, references and appendices containing the complete source code and a glossary.

# 2. Problem Statement and Objectives

## 2.1 Problem Statement

To design a 4-bit counter using Verilog HDL, to simulate it with a testbench, and to present waveform output that demonstrates its correct operation. The counter must start from a known state on reset, must count through all 16 values in the correct order, must wrap around correctly at both ends of the range, and must hold its value when counting is disabled.

## 2.2 Objectives

- To understand the working principle of a synchronous counter as a sequential circuit.
- To write clear, readable RTL code for a 4-bit counter with reset, enable and direction control.
- To write a testbench that generates the clock, applies reset and drives the control inputs.
- To carry out functional simulation and obtain waveforms that show every behaviour of the counter.
- To verify the results against the expected function table and document the observations.

## 2.3 Design Requirements

*Table 1: Design requirements of the 4-bit counter*

| Requirement | Specification |
|---|---|
| Language | Verilog HDL (VHDL is the permitted alternative) |
| Counter width | 4 bits, output range 0 to 15 (modulo-16) |
| Clock | Single clock (clk), rising-edge triggered, 10 ns period in simulation |
| Reset | Synchronous, active high (rst); forces count to 0000 |
| Control inputs | Count enable (en) and direction (up_down) |
| Output | count[3:0], registered |
| Deliverables | HDL code, functional simulation and waveform output |

# 3. Theoretical Background

## 3.1 Sequential Circuits and Counters

A combinational circuit produces outputs that depend only on the present inputs. A sequential circuit also has memory: its outputs depend on the present inputs and on a stored state. A counter is a sequential circuit whose state is simply a binary number, and whose next-state function is "add one" (or "subtract one") whenever counting is enabled.

An *n*-bit counter uses *n* flip-flops and has $2^n$ states. For n = 4 there are 16 states, from 0000 (decimal 0) to 1111 (decimal 15). Such a counter is called a **modulo-16** counter because its sequence repeats every 16 clock pulses.

## 3.2 Synchronous and Asynchronous Counters

In an **asynchronous (ripple) counter** the output of one flip-flop serves as the clock of the next, so the delays of the stages add up and intermediate states appear briefly at the outputs. In a **synchronous counter** all flip-flops share the same clock and the next-state logic decides which bits change, so every output changes at the same instant. This project uses the synchronous form, which is faster, glitch-free and the standard style for FPGA and ASIC design.

## 3.3 Modulo and Wrap-Around

Because the register is exactly 4 bits wide, adding 1 to 1111 produces a carry that is discarded and the result is 0000. Similarly, subtracting 1 from 0000 produces 1111. This overflow behaviour is the wrap-around of the counter and requires no extra logic when the register width equals the counter modulus.

## 3.4 Hardware Description Language and Verilog

A Hardware Description Language describes the structure and behaviour of digital hardware. At the Register Transfer Level (RTL) the designer specifies registers and the logic between them, and the same code can be simulated and then synthesised. The Verilog constructs used in this project are listed in Table 2.

*Table 2: Verilog constructs used*

| Construct | Purpose in this project |
|---|---|
| module / endmodule | Defines the counter as a hardware block with ports. |
| input wire / output reg | Declares port direction and type. An output assigned inside an always block is declared reg. |
| reg [3:0] | Holds the 4-bit count value. |
| always @(posedge clk) | Sequential block that runs on every rising clock edge. |
| Non-blocking assignment (<=) | Updates registers together at the clock edge, as flip-flops do. |
| if / else | Selects reset, hold, count-up or count-down according to the control inputs. |
| initial / always #delay | Used in the testbench to generate the clock and the stimulus. |
| $monitor, $dumpfile, $dumpvars | Print signal changes to the console and write a VCD waveform file. |
| `timescale 1ns / 1ps | Sets the simulation time unit (1 ns) and precision (1 ps). |

## 3.5 Synchronous Reset

In a synchronous reset the reset signal is examined only on the active clock edge. The circuit is therefore always in a well-defined state between edges, and the reset logic becomes part of the data path, which is the style recommended for FPGAs. Before the first reset edge a simulated register holds the unknown value X, which is why every register-based design needs a reset to reach a known starting state.

## 3.6 Enable and Direction Control

An enable input lets the counter pause without stopping the clock: when en is 0 the register reloads its own value and the count is held. A direction input turns the incrementer into an up/down counter, with the same register serving both directions. Together they make the counter useful as a general-purpose event counter.

## 3.7 Bit-Level View of Binary Counting

Looking at the individual bits of a binary counter shows why it also acts as a frequency divider. The least significant bit changes on every enabled clock edge, the next bit changes on every second edge, and each higher bit changes half as often as the bit below it. For a 100 MHz clock the square-wave frequency at each output bit is therefore halved at every stage, as listed in Table 2A. This property is what makes counters the basic element of clock dividers and timers.

*Table 2A: Output frequency of each bit of the counter*

| Bit | Changes every | Output frequency at 100 MHz clock | Period |
|---|---|---|---|
| count[0] | 1 clock edge | 50 MHz | 20 ns |
| count[1] | 2 clock edges | 25 MHz | 40 ns |
| count[2] | 4 clock edges | 12.5 MHz | 80 ns |
| count[3] | 8 clock edges | 6.25 MHz | 160 ns |

The same table also shows the weight of each bit: count[0] has weight 1, count[1] weight 2, count[2] weight 4 and count[3] weight 8, so the output value is count[3]x8 + count[2]x4 + count[1]x2 + count[0]x1.

# 4. Tools and Design Elements

## 4.1 Tools Used

*Table 3: Tools and environment*

| Item | Description |
|---|---|
| Language | Verilog HDL (IEEE 1364) |
| Design style | RTL behavioural modelling of a synchronous counter |
| Simulation | Verilog simulator with waveform viewer (for example Icarus Verilog with GTKWave, or Xilinx Vivado) |
| Time scale | 1 ns time unit, 1 ps precision |
| Clock | 10 ns period (100 MHz) |
| Output files | VCD waveform file, waveform images and console log |

## 4.2 Signals of the Counter

*Table 4: Signals of counter_4bit*

| Signal | Direction / type | Width | Description |
|---|---|---|---|
| clk | Input wire | 1 | System clock; all state changes occur on the rising edge |
| rst | Input wire | 1 | Synchronous reset, active high; forces count to 0000 |
| en | Input wire | 1 | Count enable; when 0 the count value is held |
| up_down | Input wire | 1 | Direction: 1 = count up, 0 = count down |
| count | Output reg | 4 | Current count value, 0 to 15 |

## 4.3 Function Table

The behaviour of the counter at every rising clock edge is summarised in Table 5. Reset has the highest priority, followed by enable, and then the direction input.

*Table 5: Function table of counter_4bit*

| rst | en | up_down | Next value of count | Operation |
|---|---|---|---|---|
| 1 | x | x | 0000 | Synchronous reset |
| 0 | 0 | x | No change | Hold |
| 0 | 1 | 1 | count + 1 (15 wraps to 0) | Count up |
| 0 | 1 | 0 | count - 1 (0 wraps to 15) | Count down |

# 5. Design of the 4-Bit Counter

## 5.1 Design Overview

The module **counter_4bit** contains a single 4-bit register and the logic in front of it. The next value of the register is chosen from four possibilities: zero (reset), the present value (hold), the present value plus one, or the present value minus one. All of this is described in one clocked always block, which synthesis tools map onto four D flip-flops with an incrementer/decrementer and a small amount of control logic.

## 5.2 Block Diagram

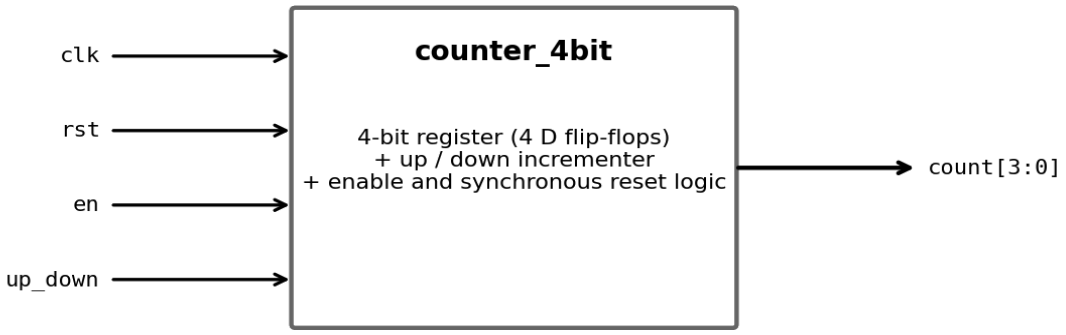



*Fig. 1: Block diagram of the 4-bit synchronous up/down counter*

The inputs clk, rst, en and up_down enter the module on the left and the registered 4-bit output count[3:0] leaves on the right. The counter has no combinational output path, so the outputs change only at clock edges.

## 5.3 Count Sequence

Table 6 lists the 16 states of the counter in binary and decimal together with the next state in each direction. The counter cycles through this table once every 16 enabled clock edges.

*Table 6: Count sequence of the 4-bit counter*

| Decimal | Binary count[3:0] | Next (up_down = 1) | Next (up_down = 0) |
|---|---|---|---|
| 0 | 0000 | 0001 (1) | 1111 (15) |
| 1 | 0001 | 0010 (2) | 0000 (0) |
| 2 | 0010 | 0011 (3) | 0001 (1) |
| 3 | 0011 | 0100 (4) | 0010 (2) |
| 4 | 0100 | 0101 (5) | 0011 (3) |
| 5 | 0101 | 0110 (6) | 0100 (4) |
| 6 | 0110 | 0111 (7) | 0101 (5) |
| 7 | 0111 | 1000 (8) | 0110 (6) |
| 8 | 1000 | 1001 (9) | 0111 (7) |
| 9 | 1001 | 1010 (10) | 1000 (8) |
| 10 | 1010 | 1011 (11) | 1001 (9) |
| 11 | 1011 | 1100 (12) | 1010 (10) |
| 12 | 1100 | 1101 (13) | 1011 (11) |
| 13 | 1101 | 1110 (14) | 1100 (12) |
| 14 | 1110 | 1111 (15) | 1101 (13) |
| 15 | 1111 | 0000 (0) | 1110 (14) |

## 5.4 Source Code Explanation

**Module header and ports.** The module declares the inputs clk, rst, en and up_down as wires and the four-bit output count as a reg, because count is assigned inside a clocked always block.

```
module counter_4bit (
    input  wire       clk,       // clock, rising-edge triggered
    input  wire       rst,       // synchronous reset (active high)
    input  wire       en,        // count enable
    input  wire       up_down,   // 1 = up, 0 = down
    output reg  [3:0] count      // 4-bit count output
);
```

*Fig. 2: Module header and port declarations*

**Sequential block.** The always block is sensitive only to the rising edge of clk, which makes the design fully synchronous. Inside it, the first condition tests rst, so reset takes priority over everything else. The second condition tests en; if en is 0 no assignment is made and the register keeps its value. When counting is enabled the up_down input selects between adding and subtracting 1.

```
always @(posedge clk) begin
    if (rst)
        count <= 4'b0000;              // reset has highest priority
    else if (en) begin
        if (up_down)
            count <= count + 4'd1;     // 15 wraps to 0
        else
            count <= count - 4'd1;     // 0 wraps to 15
    end
    // en = 0 : count holds its value
end
```

*Fig. 3: Sequential block with reset, enable and direction control*

**Non-blocking assignments.** The operator <= is used throughout so that the register behaves like a bank of flip-flops that all sample their inputs at the clock edge. **Wrap-around.** Since count is exactly four bits wide, 4'd1 is added using 4-bit arithmetic and the carry out of bit 3 is discarded, so 1111 + 1 gives 0000 and 0000 - 1 gives 1111 without any special-case code.

## 5.5 Timing Analysis

The counter changes state on each rising edge at which en is 1 and rst is 0. The testbench toggles the clock every 5 ns, so the clock period is 10 ns (100 MHz). A full pass through all 16 states therefore takes 16 x 10 ns = 160 ns, and the counter output at the most significant bit toggles at one-sixteenth of the clock frequency. Because the output is registered, it changes shortly after each rising edge and is stable for the rest of the cycle.

## 5.6 Working Steps of the Counter

- Assert rst: on the next rising edge count becomes 0000 regardless of en and up_down.
- Release rst and set en = 1: on each rising edge the counter moves to the next value.
- With up_down = 1 the sequence is 0, 1, 2, ... 15, 0, 1, ... (wrap-around from 1111 to 0000).
- With up_down = 0 the sequence is ... 2, 1, 0, 15, 14, ... (wrap-around from 0000 to 1111).
- Set en = 0: the count is held at its present value until counting is enabled again.
- Assert rst at any time: the counter returns to 0000 on the next rising edge.

## 5.7 Bit-Level Toggle Equations

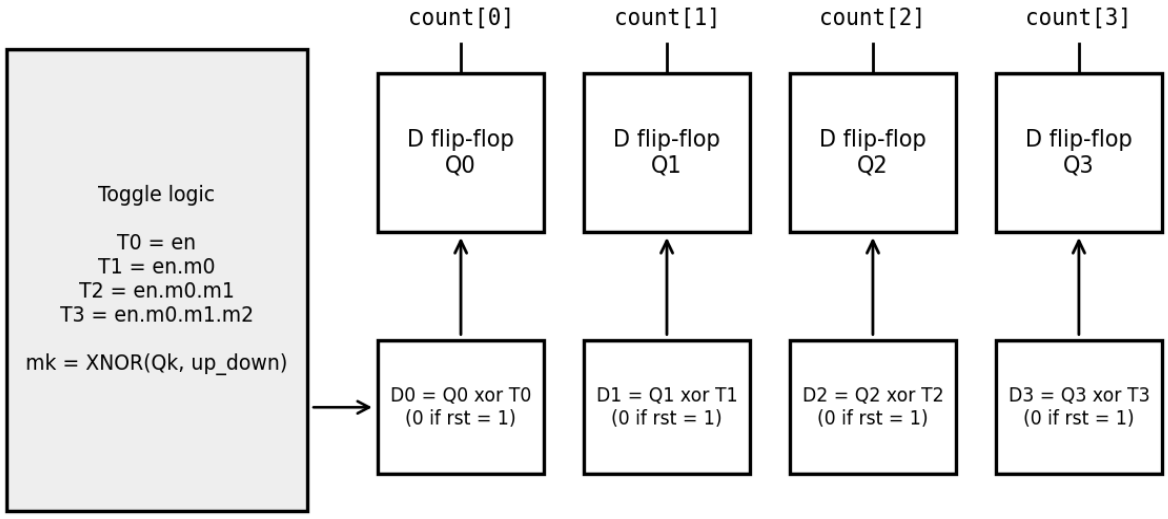
Although the Verilog code simply adds or subtracts 1, the synthesised hardware can be understood as four flip-flops whose bits toggle under simple conditions. When counting up, a bit toggles when all lower bits are 1 (a carry reaches it). When counting down, a bit toggles when all lower bits are 0 (a borrow reaches it). Writing $m_k$ = XNOR(count[k], up_down) for the statement "bit k lets the carry or borrow pass", the toggle inputs of the four bits are:

**`T0 = en, T1 = en . m0, T2 = en . m0 . m1, T3 = en . m0 . m1 . m2`**

and the next value of each bit is $D_k = 0$ when rst = 1, otherwise $D_k$ = count[k] XOR $T_k$. Table 6A lists the toggle condition of each bit in plain words.

*Table 6A: Toggle conditions of the counter bits*

| Bit | Toggles when counting up (up_down = 1) | Toggles when counting down (up_down = 0) |
|---|---|---|
| count[0] | en = 1 | en = 1 |
| count[1] | en = 1 and count[0] = 1 | en = 1 and count[0] = 0 |
| count[2] | en = 1 and count[1:0] = 11 | en = 1 and count[1:0] = 00 |
| count[3] | en = 1 and count[2:0] = 111 | en = 1 and count[2:0] = 000 |




*all flip-flops share clk; the toggle logic reads Q0..Q2, en and up_down*

*Fig. 1A: Structure of the counter as toggle logic in front of four flip-flops*

For example, counting up from 0111 to 1000: count[0], count[1] and count[2] are all 1, so T1, T2 and T3 are all 1 and every bit toggles, giving 1000. Counting down from 1000 to 0111: count[0], count[1] and count[2] are all 0, so every bit again toggles, giving 0111. These two cases are the wrap-around points of the 3-bit carry chain and are visible in the waveform of Section 6.3.

## 5.8 Corner Cases

- **Reset and enable together:** when rst = 1 the counter is cleared even if en = 1, because reset has the highest priority in the always block.
- **Reset with enable low:** reset is independent of en, so the counter still clears on the next rising edge.

- **Direction change while held:** changing up_down while en = 0 has no effect on count; the new direction applies from the next enabled edge.
- **Direction change while counting:** the counter simply continues from its present value in the new direction, as shown in phase 5 of the testbench.
- **Unknown start-up value:** before the first reset edge the simulated register holds X, and X plus or minus 1 is still X, so a reset must be applied first. The testbench does this in phase 1.
- **Maximum and minimum values:** 1111 counts up to 0000 and 0000 counts down to 1111 with no flag, because this design has no carry-out or borrow-out output.

# 6. Testbench and Simulation Results

## 6.1 Testbench

The testbench **tb_counter_4bit** instantiates the counter, generates a clock with a 10 ns period using *always #5 clk = ~clk* and drives the inputs on the falling clock edge so that they are stable well before each rising edge. The $monitor task prints every change of the signals to the console and $dumpfile / $dumpvars write a VCD file for the waveform viewer.

```
initial begin
    $dumpfile("counter_4bit.vcd");
    $dumpvars(0, tb_counter_4bit);

    rst = 1; en = 0; up_down = 1;      // 1. reset
    repeat (2)  @(negedge clk);
    rst = 0; en = 1;                   // 2. count up (wraps 15 -> 0)
    repeat (17) @(negedge clk);
    en = 0;                            // 3. hold
    repeat (3)  @(negedge clk);
    en = 1; up_down = 0;               // 4. count down (wraps 0 -> 15)
    repeat (18) @(negedge clk);
    up_down = 1;                       // 5. count up again
    repeat (5)  @(negedge clk);
    rst = 1;                           // 6. reset in the middle of counting
    @(negedge clk);
    rst = 0;                           // 7. resume counting
    repeat (2)  @(negedge clk);
    #3 $finish;
end
```

*Fig. 4: Stimulus section of the testbench (complete listing in Appendix B)*

## 6.2 Stimulus Timeline

Table 7 lists the seven stimulus phases with the absolute simulation time at which each phase begins. The complete run lasts 48 clock cycles (480 ns).

*Table 7: Testbench stimulus timeline*

| Phase | Starts at (ns) | Clock edges | rst | en | up_down | Purpose |
|---|---|---|---|---|---|---|
| 1 | 0 | 2 | 1 | 0 | 1 | Initialise the counter to 0 |
| 2 | 20 | 17 | 0 | 1 | 1 | Count up and show wrap 15 to 0 |
| 3 | 190 | 3 | 0 | 0 | 1 | Check hold behaviour |
| 4 | 220 | 18 | 0 | 1 | 0 | Count down and show wrap 0 to 15 |
| 5 | 400 | 5 | 0 | 1 | 1 | Check direction change |
| 6 | 450 | 1 | 1 | 1 | 1 | Reset while counting |
| 7 | 460 | 2 | 0 | 1 | 1 | Check counting resumes |

## 6.3 Simulation Waveform

Figure 5 shows the complete waveform for all 48 clock cycles. The count value is shown in hexadecimal in each cycle; the red X before the first clock edge is the unknown initial value of the register. Figure 6 zooms into the first 22 cycles to show reset release, counting from 1 to F and the wrap-around to 0.

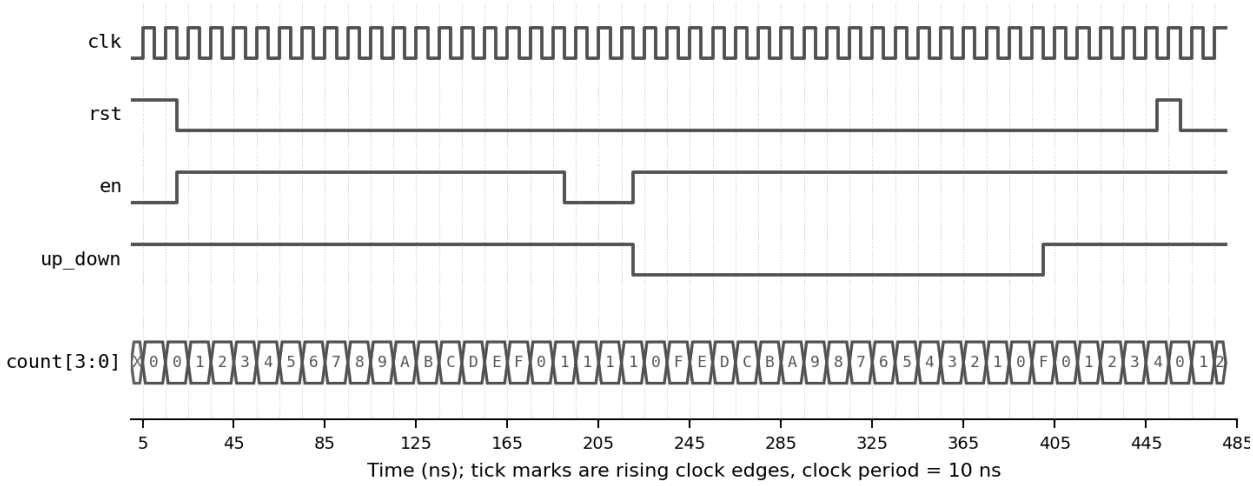



*Fig. 5: Simulation waveform of count[3:0] with clk, rst, en and up_down (complete run)*

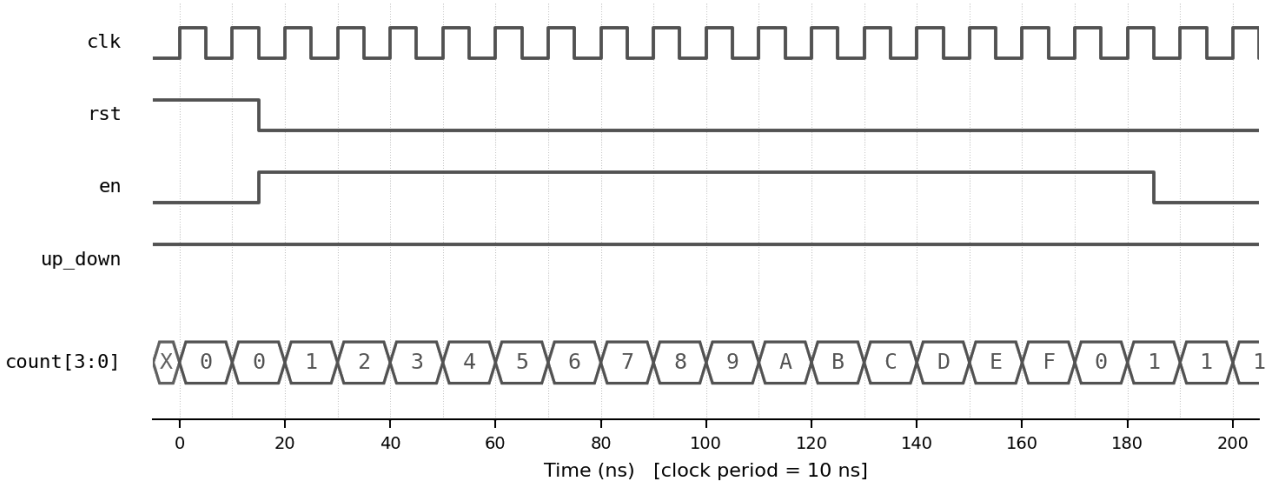



*Fig. 6: Zoomed view: reset release, count up from 1 to F, wrap-around to 0*

The waveform shows the output holding 0 while rst is high, counting 1, 2, ... F and then rolling over to 0 when counting up, staying constant while en is low, counting down through 0 to F when up_down is low, and returning to 0 when rst is asserted in the middle of the run.

## 6.4 Console Output

The $monitor statement prints a line whenever rst, en, up_down or count changes. The first lines of the log, including the first wrap-around, and the lines for the final phases are shown below.

```
t=0   ns | rst=1 en=0 up_down=1 | count= x (xxxx)
t=5   ns | rst=1 en=0 up_down=1 | count= 0 (0000)
t=20  ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=25  ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=35  ns | rst=0 en=1 up_down=1 | count= 2 (0010)
t=45  ns | rst=0 en=1 up_down=1 | count= 3 (0011)
   ...
t=155 ns | rst=0 en=1 up_down=1 | count=14 (1110)
t=165 ns | rst=0 en=1 up_down=1 | count=15 (1111)
t=175 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=185 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=190 ns | rst=0 en=0 up_down=1 | count= 1 (0001)
   ...
t=220 ns | rst=0 en=1 up_down=0 | count= 1 (0001)
t=225 ns | rst=0 en=1 up_down=0 | count= 0 (0000)
t=235 ns | rst=0 en=1 up_down=0 | count=15 (1111)
t=245 ns | rst=0 en=1 up_down=0 | count=14 (1110)
   ...
t=395 ns | rst=0 en=1 up_down=0 | count=15 (1111)
t=400 ns | rst=0 en=1 up_down=1 | count=15 (1111)
t=405 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=415 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=425 ns | rst=0 en=1 up_down=1 | count= 2 (0010)
t=435 ns | rst=0 en=1 up_down=1 | count= 3 (0011)
t=445 ns | rst=0 en=1 up_down=1 | count= 4 (0100)
t=450 ns | rst=1 en=1 up_down=1 | count= 4 (0100)
t=455 ns | rst=1 en=1 up_down=1 | count= 0 (0000)
t=460 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=465 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=475 ns | rst=0 en=1 up_down=1 | count= 2 (0010)
Simulation finished at t=483 ns ($finish)
```

*Fig. 7: Expected $monitor console log (selected lines)*

## 6.5 Edge-by-Edge Trace

Using the stimulus timeline and the function table, the behaviour of the counter can be traced edge by edge. Rising edges occur at 5, 15, 25, ... ns (5 + 10k ns), and the count is read just after each edge.

*Table 8: Edge-by-edge trace of counter_4bit*

| Edges | Time (ns) | rst en up | count after edge | Comment |
|---|---|---|---|---|
| 0 to 1 | 5 to 15 | 1 0 1 | 0 to 0 | Reset applied: X becomes 0000 |
| 2 | 25 | 0 1 1 | 1 (0001) | rst released, en = 1: first count up |
| 3 to 16 | 35 to 165 | 0 1 1 | 2 to 15 | Counting up to the maximum value F |
| 17 | 175 | 0 1 1 | 0 (0000) | Wrap-around: 1111 to 0000 |
| 18 | 185 | 0 1 1 | 1 (0001) | Counting continues |
| 19 to 21 | 195 to 215 | 0 0 1 | 1 to 1 | en = 0: count held |
| 22 | 225 | 0 1 0 | 0 (0000) | en = 1, down: 1 to 0 |
| 23 | 235 | 0 1 0 | 15 (1111) | Wrap-around: 0000 to 1111 |
| 24 to 39 | 245 to 395 | 0 1 0 | 14 to 15 | Counting down |
| 40 | 405 | 0 1 1 | 0 (0000) | up_down = 1: wrap 1111 to 0000 |
| 41 to 44 | 415 to 445 | 0 1 1 | 1 to 4 | Counting up again |
| 45 | 455 | 1 1 1 | 0 (0000) | rst = 1: reset to 0000 |
| 46 to 47 | 465 to 475 | 0 1 1 | 1 to 2 | rst released: counting resumes |

## 6.6 Verification Summary

Table 9 compares the expected behaviour with the behaviour observed in the waveform for each test.

*Table 9: Verification of counter_4bit*

| Phase | Test | Expected | Observed in waveform | Result |
|---|---|---|---|---|
| 1 | Reset | count = 0 after first edge with rst = 1 | count = 0000 | Pass |
| 2 | Count up | 0, 1, 2, ... 15 | Increments by 1 each edge | Pass |
| 2 | Up wrap-around | 1111 followed by 0000 | Rolls over to 0 | Pass |
| 3 | Hold (en = 0) | count unchanged | Value held for 3 cycles | Pass |
| 4 | Count down | ... 2, 1, 0 | Decrements by 1 each edge | Pass |
| 4 | Down wrap-around | 0000 followed by 1111 | Rolls over to F | Pass |
| 5 | Direction change | Resumes counting up | Increments again | Pass |
| 6 | Reset during count | count = 0 on next edge | count = 0000 | Pass |
| 7 | Resume | 1, 2 after reset release | Counts 1, 2 | Pass |

## 6.7 Summary of Results

- The counter starts from a known value (0000) after reset and counts in the correct order.
- All 16 states appear in sequence in both directions, with correct wrap-around at 15 and 0.
- The output holds its value when en is low and ignores up_down while held.
- Reset takes priority over enable and direction, even in the middle of counting.
- The output changes only after rising clock edges, confirming synchronous operation.

## 6.8 Bit-Level Waveform and Frequency Division

Figure 7A shows the individual bits of count[3:0] during the first 36 clock cycles of counting up. count[0] changes at every clock edge, count[1] at every second edge, count[2] at every fourth and count[3] at every eighth, which confirms the frequency-division property listed in Table 2A and the toggle conditions in Table 6A. After 16 edges the pattern repeats because the counter is modulo-16.

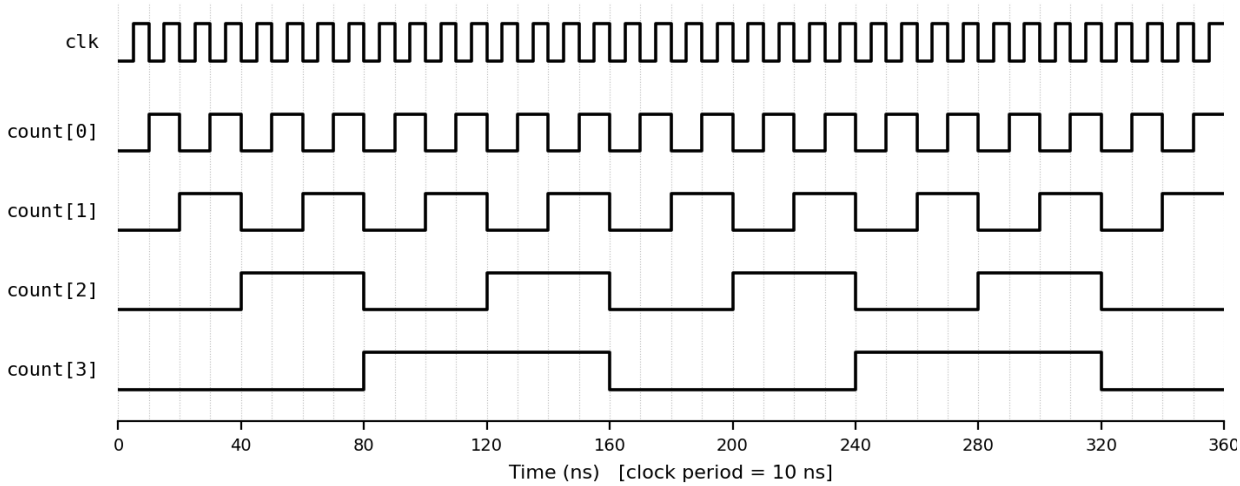



*Fig. 7A: Individual bits of the counter during 36 clock cycles of counting up*

# 7. Comparison of Counter Types

The counter in this project is a synchronous binary counter. Table 10 compares it with the other common counter styles so that the design choice is clear.

*Table 10: Comparison of counter types*

| Parameter | Asynchronous (ripple) | Synchronous binary (this project) | Ring / Johnson |
|---|---|---|---|
| Clocking | Output of one stage clocks the next | All flip-flops share one clock | All flip-flops share one clock |
| Speed | Slow, delays accumulate | Fast, limited by one stage plus logic | Fast, no extra logic |
| Glitches | Transient wrong states | None at registered outputs | None |
| States from 4 flip-flops | 16 | 16 | 4 (ring) or 8 (Johnson) |
| Extra logic | Almost none | Incrementer / decrementer | None, only feedback |
| Typical use | Simple frequency dividers | General counting, addressing, timing | Sequencers, one-hot control |

The binary synchronous counter is the best general-purpose choice: it uses all $2^n$ states of the register, has clean outputs and is easy to extend with enable, direction, load and terminal-count features. A ripple counter saves logic but is unsuitable when timing must be analysed, and ring or Johnson counters trade state count for simpler decoding.

# 8. Advantages, Limitations, Applications and Future Scope

## 8.1 Advantages

- The design is short, readable and described by a single clocked always block.
- Synchronous operation gives glitch-free outputs and predictable timing.
- Enable and direction inputs make the counter flexible without extra hardware blocks.
- Synchronous reset brings the counter to a known state and maps well to FPGA flip-flops.
- The testbench covers every function of the counter, so the behaviour is completely verified.

## 8.2 Limitations

- The counter is limited to 4 bits, so it counts only up to 15 before wrapping.
- There is no parallel-load input and no terminal-count (carry-out) signal.
- Verification is by simulation only; the design has not been synthesised or tested on hardware.
- The testbench is directed and checks results by waveform inspection rather than automatic comparison.

## 8.3 Applications

- Event counting and timers; clock dividers and frequency counters.
- Address generation for memories and register files.
- Sequencing and control logic, and as the timing element in finite state machines.
- Teaching digital design, sequential logic and Verilog HDL.

## 8.4 Future Scope

- Parameterise the counter width so the same module can be used for 8, 16 or 32 bits.
- Add a parallel-load input and a terminal-count output for cascading counters.
- Add a BCD (modulo-10) mode and a programmable modulus.
- Write a self-checking testbench with assertions and compare against a reference model.
- Synthesise the design, review area and timing reports, and implement it on an FPGA board with LEDs.

# 9. Conclusion

This project presented the design and simulation of a 4-bit counter in Verilog HDL. The counter is synchronous, with a synchronous active-high reset, a count-enable input and an up/down direction input. It counts through all 16 values in the correct order, wraps around from 15 to 0 when counting up and from 0 to 15 when counting down, holds its value when disabled and returns to 0000 whenever reset is asserted.

A testbench with seven stimulus phases exercised every function of the design, and the waveforms, the console log and the edge-by-edge trace agree with the function table in every phase, so the design is functionally correct. Through the theory, code explanation, simulation and verification tables, the report shows how clocked registers, non-blocking assignments, synchronous control signals and testbench-based verification combine to describe real hardware.

The work reinforced several RTL design practices: keeping all state changes on one clock edge, giving reset the highest priority, relying on exact register width for wrap-around, and verifying behaviour with a testbench and a waveform viewer. The suggested enhancements, such as a parameterised width, parallel load, terminal count and FPGA implementation, offer a clear path to extend this learning project.

# References

[1] Unlox Academy, Minor Project 03: Design and Simulation of a 4-Bit Counter, VLSI program assignment (Minor_Project_03_September_VLSI), September 2026.

[2] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.

[3] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed., Prentice Hall.

[4] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, Pearson.

# Appendix A: Complete Source Code of the Counter

The listing below is the complete module counter_4bit (file counter_4bit.v).

```
`timescale 1ns / 1ps
//------------------------------------------------------------
// Module : counter_4bit
// Brief  : 4-bit synchronous up/down counter
//          - synchronous active-high reset
//          - count enable
//          - up_down = 1 : count up, up_down = 0 : count down
//------------------------------------------------------------
module counter_4bit (
    input  wire       clk,       // clock, rising-edge triggered
    input  wire       rst,       // synchronous reset (active high)
    input  wire       en,        // count enable
    input  wire       up_down,   // 1 = up, 0 = down
    output reg  [3:0] count      // 4-bit count output
);

    always @(posedge clk) begin
        if (rst)
            count <= 4'b0000;              // reset has highest priority
        else if (en) begin
            if (up_down)
                count <= count + 4'd1;     // 15 wraps to 0
            else
                count <= count - 4'd1;     // 0 wraps to 15
        end
        // en = 0 : count holds its value
    end

endmodule
```

# Appendix B: Complete Testbench

The listing below is the complete testbench tb_counter_4bit (file tb_counter_4bit.v). To simulate with Icarus Verilog and GTKWave:

```
iverilog -o sim counter_4bit.v tb_counter_4bit.v
vvp sim
gtkwave counter_4bit.vcd

`timescale 1ns / 1ps
module tb_counter_4bit;

    reg        clk, rst, en, up_down;
    wire [3:0] count;

    // Device under test
    counter_4bit dut (.clk(clk), .rst(rst), .en(en),
                      .up_down(up_down), .count(count));

    // 10 ns clock (100 MHz)
    initial clk = 1'b0;
    always  #5 clk = ~clk;

    // Console monitor
    initial $monitor("t=%0d ns | rst=%b en=%b up_down=%b | count=%2d (%b)",
                     $time, rst, en, up_down, count, count);

    // Stimulus - inputs change on the falling edge
    initial begin
        $dumpfile("counter_4bit.vcd");
        $dumpvars(0, tb_counter_4bit);

        rst = 1; en = 0; up_down = 1;      // 1. reset
        repeat (2)  @(negedge clk);
        rst = 0; en = 1;                   // 2. count up (wraps 15 -> 0)
        repeat (17) @(negedge clk);
        en = 0;                            // 3. hold
        repeat (3)  @(negedge clk);
        en = 1; up_down = 0;               // 4. count down (wraps 0 -> 15)
        repeat (18) @(negedge clk);
        up_down = 1;                       // 5. count up again
        repeat (5)  @(negedge clk);
        rst = 1;                           // 6. reset in the middle of counting
        @(negedge clk);
        rst = 0;                           // 7. resume counting
        repeat (2)  @(negedge clk);
        #3 $finish;
    end

endmodule
```

# Appendix C: Glossary

| Term | Meaning |
|---|---|
| Counter | Sequential circuit whose state is a binary number that changes on each clock pulse. |
| Modulo-16 | A counter that repeats after 16 states (0 to 15), as a 4-bit counter does. |
| Synchronous | All state changes happen on the same clock edge. |
| Wrap-around | Roll-over from the maximum value to 0 (or 0 to the maximum) because of fixed register width. |
| RTL | Register Transfer Level: description of hardware as registers and the logic between them. |
| HDL | Hardware Description Language such as Verilog or VHDL. |
| Testbench | Simulation-only module that supplies the clock, reset and inputs to the design under test. |
| Non-blocking assignment | Assignment (<=) that updates registers together at the end of the time step; used for sequential logic. |
| Synchronous reset | Reset that is sampled only on the active clock edge. |
| VCD | Value Change Dump: file format that stores signal changes for waveform viewers. |
| Time scale | Directive defining the unit and precision of simulation time. |

# Appendix D: Complete Console Log

The listing below is the complete expected $monitor output of tb_counter_4bit. A line is printed at time 0 and whenever rst, en, up_down or count changes; inputs change at falling clock edges (10, 20, ... ns) and count changes just after rising edges (5, 15, 25, ... ns).

```
t=0   ns | rst=1 en=0 up_down=1 | count= x (xxxx)
t=5   ns | rst=1 en=0 up_down=1 | count= 0 (0000)
t=20  ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=25  ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=35  ns | rst=0 en=1 up_down=1 | count= 2 (0010)
t=45  ns | rst=0 en=1 up_down=1 | count= 3 (0011)
t=55  ns | rst=0 en=1 up_down=1 | count= 4 (0100)
t=65  ns | rst=0 en=1 up_down=1 | count= 5 (0101)
t=75  ns | rst=0 en=1 up_down=1 | count= 6 (0110)
t=85  ns | rst=0 en=1 up_down=1 | count= 7 (0111)
t=95  ns | rst=0 en=1 up_down=1 | count= 8 (1000)
t=105 ns | rst=0 en=1 up_down=1 | count= 9 (1001)
t=115 ns | rst=0 en=1 up_down=1 | count=10 (1010)
t=125 ns | rst=0 en=1 up_down=1 | count=11 (1011)
t=135 ns | rst=0 en=1 up_down=1 | count=12 (1100)
t=145 ns | rst=0 en=1 up_down=1 | count=13 (1101)
t=155 ns | rst=0 en=1 up_down=1 | count=14 (1110)
t=165 ns | rst=0 en=1 up_down=1 | count=15 (1111)
t=175 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=185 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=190 ns | rst=0 en=0 up_down=1 | count= 1 (0001)
t=220 ns | rst=0 en=1 up_down=0 | count= 1 (0001)
t=225 ns | rst=0 en=1 up_down=0 | count= 0 (0000)
t=235 ns | rst=0 en=1 up_down=0 | count=15 (1111)
t=245 ns | rst=0 en=1 up_down=0 | count=14 (1110)
t=255 ns | rst=0 en=1 up_down=0 | count=13 (1101)
t=265 ns | rst=0 en=1 up_down=0 | count=12 (1100)
t=275 ns | rst=0 en=1 up_down=0 | count=11 (1011)
t=285 ns | rst=0 en=1 up_down=0 | count=10 (1010)
t=295 ns | rst=0 en=1 up_down=0 | count= 9 (1001)
t=305 ns | rst=0 en=1 up_down=0 | count= 8 (1000)
t=315 ns | rst=0 en=1 up_down=0 | count= 7 (0111)
t=325 ns | rst=0 en=1 up_down=0 | count= 6 (0110)
t=335 ns | rst=0 en=1 up_down=0 | count= 5 (0101)
t=345 ns | rst=0 en=1 up_down=0 | count= 4 (0100)
t=355 ns | rst=0 en=1 up_down=0 | count= 3 (0011)
t=365 ns | rst=0 en=1 up_down=0 | count= 2 (0010)
t=375 ns | rst=0 en=1 up_down=0 | count= 1 (0001)
t=385 ns | rst=0 en=1 up_down=0 | count= 0 (0000)
t=395 ns | rst=0 en=1 up_down=0 | count=15 (1111)
t=400 ns | rst=0 en=1 up_down=1 | count=15 (1111)
t=405 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=415 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=425 ns | rst=0 en=1 up_down=1 | count= 2 (0010)
t=435 ns | rst=0 en=1 up_down=1 | count= 3 (0011)
t=445 ns | rst=0 en=1 up_down=1 | count= 4 (0100)
t=450 ns | rst=1 en=1 up_down=1 | count= 4 (0100)
t=455 ns | rst=1 en=1 up_down=1 | count= 0 (0000)
t=460 ns | rst=0 en=1 up_down=1 | count= 0 (0000)
t=465 ns | rst=0 en=1 up_down=1 | count= 1 (0001)
t=475 ns | rst=0 en=1 up_down=1 | count= 2 (0010)
Simulation finished at t=483 ns ($finish)
```

# Appendix E: Submission Checklist

The assignment requires all project files (code and simulation results) to be uploaded to Google Drive or a GitHub repository, with the shareable link submitted. The link must be accessible with no restricted access, the submission is individual, and a proper file structure must be maintained. The folder structure and checklist used for this project are given below.

```
Minor_Project_03_4bit_Counter/
|-- src/
|     |-- counter_4bit.v
|-- tb/
|     |-- tb_counter_4bit.v
|-- sim/
|     |-- counter_4bit.vcd
|     |-- waveform_full.png
|     |-- waveform_zoom.png
|-- report/
|     |-- Minor_Project_03_4-Bit_Counter_Report.pdf
|-- README.md
```

| Requirement | Where it is in this project |
|---|---|
| HDL code (Verilog) for the 4-bit counter | Chapter 5 and Appendix A |
| Functional simulation | Chapter 6 and Appendices B and D |
| Waveform output demonstrating correct operation | Section 6.3 (Figures 5 and 6) and Section 6.8 |
| All files uploaded to Google Drive or GitHub | To be uploaded by the student |
| Shareable link with unrestricted access | Link: _____ |
| Individual submission | Yes |
| Proper file structure maintained | As shown above |

# Appendix F: Simulation Procedure

The design and testbench can be simulated with any standard Verilog simulator. The two common flows used for this project are described below.

## F.1 Icarus Verilog with GTKWave

```
iverilog -o sim counter_4bit.v tb_counter_4bit.v
vvp sim
gtkwave counter_4bit.vcd
```

- The first command compiles the design and testbench into the executable sim.
- The second command runs the simulation; the $monitor lines appear on the console and the file counter_4bit.vcd is written.
- The third command opens the VCD file in GTKWave; add clk, rst, en, up_down and count[3:0] to the waveform window and set count to hexadecimal display.

## F.2 Xilinx Vivado, ModelSim or Questa

- Create a new project and add counter_4bit.v as a design source.
- Add tb_counter_4bit.v as a simulation source and set it as the top simulation module.
- Run behavioural simulation and run for about 500 ns.
- Add all signals to the waveform window, set the radix of count to hexadecimal and use Zoom Fit to view the complete run.
- Save the waveform screenshots in the sim folder together with the VCD file.

## F.3 What to Check in the Waveform

- count is X until the first rising clock edge and 0 after the first edge with rst = 1.
- count increases by 1 at every rising edge while en = 1 and up_down = 1, and rolls over from F to 0.
- count stays unchanged while en = 0.
- count decreases by 1 at every rising edge while en = 1 and up_down = 0, and rolls over from 0 to F.
- count returns to 0 on the first rising edge after rst is asserted, even in the middle of counting.
- All changes of count occur just after rising clock edges.